

7.5. Almacenamiento

Antes de HTML5, solo se podía guardar información en el navegador del cliente (como preferencias o el idioma predeterminado para una web) utilizando **cookies**.

Con HTML5, se incorporó la **API Storage** para mejorar esta funcionalidad.



También existen métodos más avanzados de almacenamiento en el cliente, como **IndexedDB**, que es un estándar del W3C.

7.5.1. Cookies

Pequeños fragmentos de datos que se almacenan en el navegador del usuario y se utilizan para guardar información sobre el mismo y su interacción con un sitio web.

Tienen algunas limitaciones:

- **Máximo de cookies por dominio:** si se excede este límite, las más antiguas se eliminan. Aproximadamente:
 - Chrome: 180.
 - Firefox: 150.
 - Safari: 50.

→ El límite total de cookies del navegador suele ser de miles.

- **Tamaño máximo** de 4 KB **por cookie**: si se supera este tamaño, la cookie se truncará.

Pero, son soportadas por navegadores antiguos.

Formato

Cada cookie almacena los siguientes datos:

- **Nombre** de la cookie (obligatorio).
- **Valor** de la cookie.

- **expires:** marca de tiempo en la que se borrará (si no se especifica, se eliminará al salir del dominio).
- **max-age:** en lugar de 'expires', podemos indicar aquí los segundos que durará la cookie antes de expirar.
- **path:** ruta desde la que es accesible (/: todo el dominio, /xxx: esa carpeta y subcarpetas). Si no se especifica, solo será accesible desde la carpeta actual.
- **domain:** dominio desde el que es accesible. Si no se especifica, será accesible desde este dominio y sus subdominios.
- **secure:** si está presente, indica que la cookie solo se enviará a través de HTTPS.

Acceso

Se puede acceder a las cookies desde **document.cookie** que es una cadena con la información de las cookies de nuestra web.

Por ejemplo:

```
document.cookie = "username=Juanan; expires=Fri, 31 Dec 2025 23:59:59 GMT; path="/";
```



```
document.cookie = "username=" + encodeURIComponent("Juanan García") +  
"; expires=Fri, 31 Dec 2025 23:59:59 GMT; path="/";
```

→ Las cookies no permiten espacios sin codificar

Podemos definirnos una función para la creación de cookies ...

```
function setCookie(cname, cvalue, cexpires, cpath, cdomain, csecure) {  
    document.cookie = cname + '=' + cvalue +  
        (cexpires ? ';expires=' + cexpires.toUTCString() : '') +  
        (cpath ? ';path=' + cpath : '') +  
        (cdomain ? ';domain=' + cdomain : '') +  
        (csecure ? ';secure' : '');  
}
```



```
// Crear una cookie que expira en una fecha específica  
const expirationDate = new Date();  
expirationDate.setDate(expirationDate.getDate() + 7); // Expira en 7 días  
setCookie('username', 'Juanan', expirationDate, '/', '', true);
```

```
// Crear una cookie de sesión (sin fecha de expiración)  
setCookie('sessionID', 'abc123');
```

```
// Modifica una cookie  
expirationDate.setDate(expirationDate.getDate() + 21); // Expira en 21 días más = 28  
setCookie('username', 'Juanan', expirationDate, '/', '', true);
```

Otra función para la extracción del valor de una cookie ...

```
function getCookie(cname) {  
    if (document.cookie.length > 0) {  
        let start = document.cookie.indexOf(cname + '=');  
        if (start !== -1) { // Existe la cookie  
            start = start + cname.length + 1; // Inicio del valor  
            let end = document.cookie.indexOf(';', start);  
            if (end === -1) { // Si no encuentra ';', es la última cookie  
                end = document.cookie.length;  
            }  
            return document.cookie.substring(start, end); // Extrae el valor  
        }  
    }  
    return ''; // No se encontró la cookie  
}
```

```
console.log(getCookie('username'));  
console.log(getCookie('noexiste'));  
console.log(getCookie('sessionID'));
```

Para ver todas las cookies de la página web ...

```
// document.cookie contiene todas las cookies del sitio  
console.log('Cookies: ' + document.cookie);
```

```
Cookies: sessionID=abc123; username=Juanan
```

Para eliminar una Cookie se ha de configurar el parámetro de caducidad en una fecha pasada ...

```
function delCookie(cname) {  
    document.cookie = cname + ';;expires=Thu, 01 Jan 1970 00:00:01 GMT;'  
    return document.cookie;  
}
```



```
console.log('Cookie sessionId: ' + getCookie('sessionId'));  
deleteCookie('sessionId'); // Elimina la cookie  
console.log('Cookie sessionId: ' + getCookie('sessionId'));
```

Cookie sessionId: abc123

Cookie sessionId:

Más información:

<https://developer.mozilla.org/es/docs/Web/API/Document/cookie>

7.5.2. Vite

Cuando estudiamos los módulos vimos las tareas que desempeñan los automatizadores ...

- Creación y gestión del proyecto.
- Actualización y mantenimiento de dependencias.
- Servidor web local de desarrollo.
- Minificación de código.
- Optimización de imágenes.
- Preprocesamiento o transpilación del código fuente.
- Despliegue del código final al servidor.

Actualmente, **Vite** el más recomendado y utilizado.

Es una de las herramientas de **construcción de proyectos** o module bundlers (empaquetadores de módulos) más utilizados actualmente en entorno **frontend**.

Incorpora un servidor de desarrollo para hacer más cómoda la creación y prueba de proyectos.

Para usar Vite, se ha de instalar:



```
npm install -g vite
```

Para crear un nuevo proyecto:

npm create vite@latest

Al crear el proyecto nos pregunta qué framework vamos a utilizar (Vanilla-Javascript sin framework) y si como lenguaje usaremos Javascript o Typescript.



```
◆ Select a framework:
```

- Vanilla
- Vue
- React
- Preact
- Lit
- Svelte
- Solid
- Qwik
- Angular
- Marko
- Others

```
◆ Select a variant:
```

- TypeScript
- JavaScript

Esto crea el **scaffolding** de nuestro proyecto que consiste en una carpeta con el mismo nombre que el proyecto y una serie de ficheros en su interior.

```
◇ Select a framework:
  Vanilla

◇ Select a variant:
  JavaScript

◇ Use rolldown-vite (Experimental)?:
  No

◆ Install with npm and start now?
  ● Yes / ○ No
```

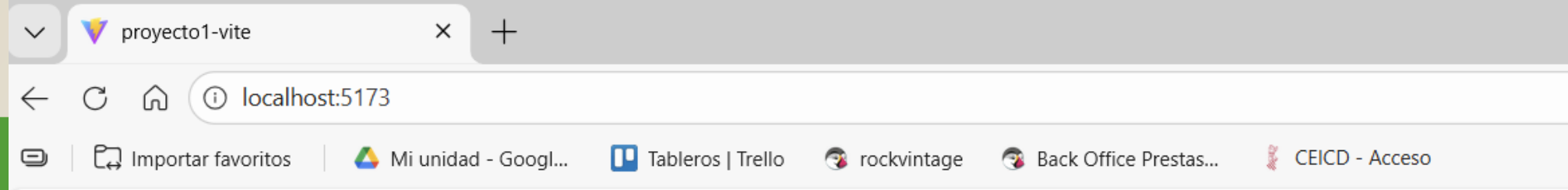
```
◇ Starting dev server...

> ejemplo-vite@0.0.0 dev
> vite

VITE v7.3.1 ready in 335 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

```
✓ ejemplo-vite
  > node_modules
  > public
  > src
  > .gitignore
  > index.html
  > package-lock.json
  > package.json
```

→ Se lanza un servidor web en el puerto **5173** donde podemos ver la ejecución del proyecto.



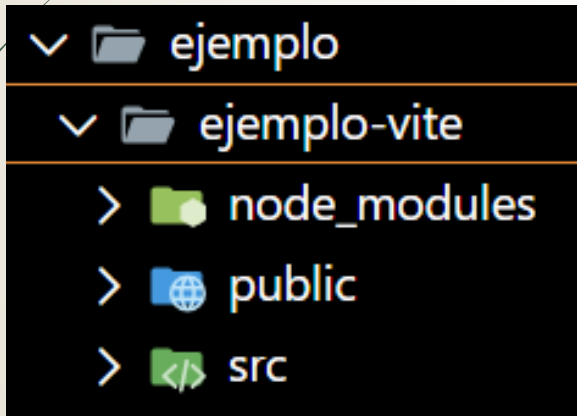
Hello Vite!

count is 0

[Click on the Vite logo to learn more](#)

Tras esta instalación y primera prueba de ejecución, si detenemos el servidor con **ctrl + c** y más tarde queremos volver a arrancarlo tenemos que:

- Situarnos en la carpeta del proyecto vite (no la del padre, desde la que hicimos la creación)



- Arrancar el servidor con:
npm run dev

```
> ejemplo-vite@0.0.0 dev
> vite

VITE v7.3.1 ready in 302 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Hay un archivo opcional en el directorio raíz, **vite.config.js**, que sirve para configurar **vite** y decirle cosas como:

- en qué carpeta está el código
- cómo se hace el build
- base de la aplicación
- plugins
- alias de rutas

→ Ahora mismo no nos hace falta y no vamos a entrar más en detalle sobre este archivo.

Un proyecto vite podría ser este ...

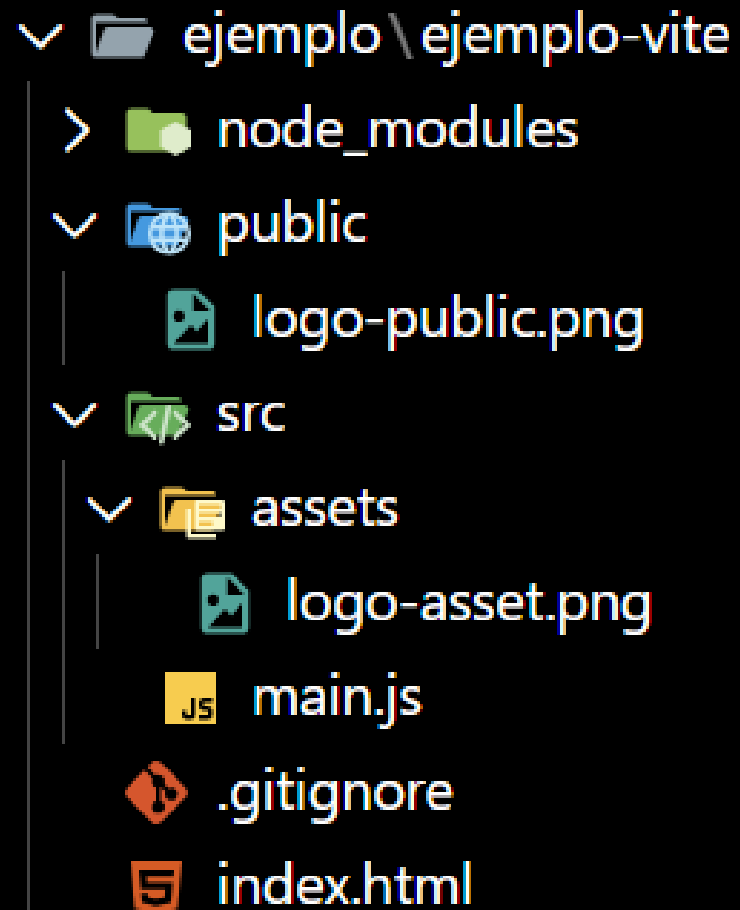
```
ejemplo-vite/  
├─ index.html  
├─ public/  
│   └─ logo-public.png  
├─ src/  
│   ├── main.js  
│   └─ assets/  
│       └─ logo-asset.png
```

- Todo lo que está en **public** se referencia como si estuviera en la raíz de nuestra aplicación.

```
const img = document.createElement('img')  
img.src = '/logo-public.png'
```

- Lo que tengamos en la carpeta **assets**, antes de usarlo deberemos importarlo (ruta relativa).

```
import logoAsset from './assets/logo-asset.png'
```





Ejemplo Vite



APP Docents - Genera

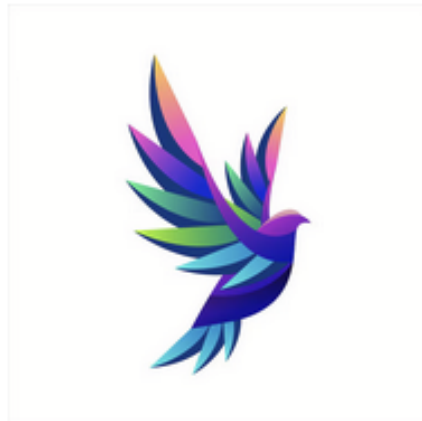


http://localhost:5173

Imagen desde public



Imagen desde assets



Cuando se quiere pasar a producción, se ha de ejecutar ...

```
npm run build
```

→ crea la carpeta **/dist** con los ficheros que debemos subir al servidor web de producción:

- **index.html**
- cualquier fichero que tengamos en **/public**
- carpeta **assets** con:
 - **fichero JS** con todo el código que necesita el proyecto
 - **fichero CSS** con todos los estilos del proyecto
 - **otros ficheros importados** en el JS como imágenes, ...

```
> ejemplo-vite@0.0.0 build
> vite build
```

```
vite v7.3.1 building client environment for production...
```

```
✓ 4 modules transformed.
```

dist/index.html	0.50 kB		gzip: 0.34 kB
dist/assets/logo-asset-Bq-CM8Uu.png	9.56 kB		
dist/assets/index-BLZ8bU14.js	0.86 kB		gzip: 0.48 kB
✓ built in 184ms			



```
✓ ejemplo \ ejemplo-vite
  ✓ dist
    ✓ assets
      index-BLZ8bU14.js
      logo-asset-Bq-CM8Uu.png
    index.html
    logo-public.png
```

Para previsualizar la ejecución de producción, se ha de ejecutar:

```
npm run preview
```

```
> ejemplo-vite@0.0.0 preview
```

```
> vite preview
```

```
→ Local:   http://localhost:4173/
```

```
→ Network: use --host to expose
```

```
→ press h + enter to show help
```

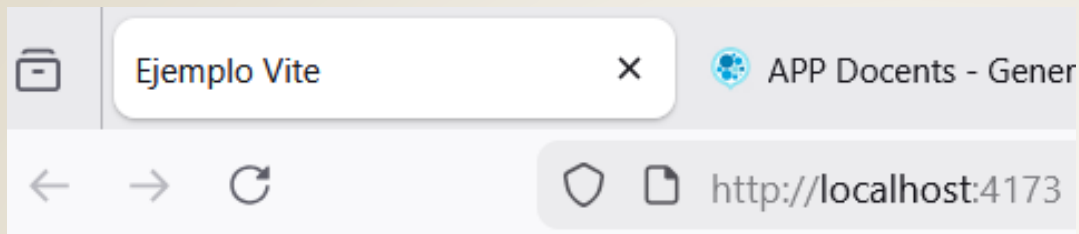


Imagen desde public



Imagen desde assets



Más información:

<https://vite.dev/guide/why.html>

Testing

Es fundamental que nuestro código tenga un cierto nivel de calidad para minimizar los fallos del programa, más cuanto más compleja es la aplicación que estamos desarrollando.

Para ello debemos testearlo y dicho testeo seguramente incluirá test automáticos.

Tenemos muchas herramientas para hacer tests unitarios, como Jest o Vitest.

<https://jestjs.io/>

<https://vitest.dev/>

7.5.3. API Storage

La API Storage proporciona, dentro del objeto **window**, los objetos **localStorage** y **sessionStorage**.

En ellos se guarda información en el almacenamiento local: de 5 y 10 MB por sitio web, dependiendo del navegador, mucho más que con Cookies.

La diferencia entre los objetos es que la información almacenada en **localStorage** nunca expira; permanece allí hasta que se elimine.

En cambio, la información almacenada en **sessionStorage**, se elimina automáticamente al cerrar la página.

Para verificar si el navegador la soporta, podemos comprobar el tipo:

```
if (typeof(Storage) === 'undefined') // NO está soportado
```

Tanto **localStorage** como **sessionStorage** son objetos globales accesibles desde el código:

- Guardar un dato:

```
localStorage.setItem('dato', 'valor')  
  
localStorage.dato = 'valor';
```

- Recuperar un dato:

```
let miDato = localStorage.getItem('dato')
```

```
let miDato = localStorage.dato;
```

- Borrar datos:

```
localStorage.removeItem('dato');
```

```
localStorage.clear(); // borrar todo
```

- Conocer cantidad datos:

```
localStorage.length;
```

Podríamos ver todos los datos así ...

```
for (let i = 0; i < localStorage.length; i++) {  
  const clave = localStorage.key(i);  
  const valor = localStorage.getItem(clave);  
  console.log(clave, valor);  
}
```

→ Sólo se pueden guardar objetos primitivos (cadenas, números, etc.). Para guardar un objeto o array, hay que convertirlo a JSON:

```
localStorage.setItem('dato', JSON.stringify(objeto));
```

→ Para recuperar el objeto:

```
let miObjeto= JSON.parse(localStorage.getItem('dato'));
```

Cada vez que cambia la información en **localStorage** se produce un **evento storage**.

→ Para que una ventana actualice su información cuando otra cambia un dato del **storage** ...

```
window.addEventListener("storage", function(){});
```

Ejemplo sencillo (contador) ...

/index.html

```
<body>
  <div id="app"></div>
  <script type="module" src="/src/main.js"></script>
</body>
```

/src/main.js

```
// Obtener valor guardado o usar 0 por defecto
let contador = Number(localStorage.getItem("contador")) || 0;

const app = document.querySelector("#app");

const render = () => {
  app.innerHTML = `
    <h1>Contador: ${contador}</h1>
    <button id="sumar">+ Sumar</button>
    <button id="reset">↺ Reset</button>
  `;
};
```

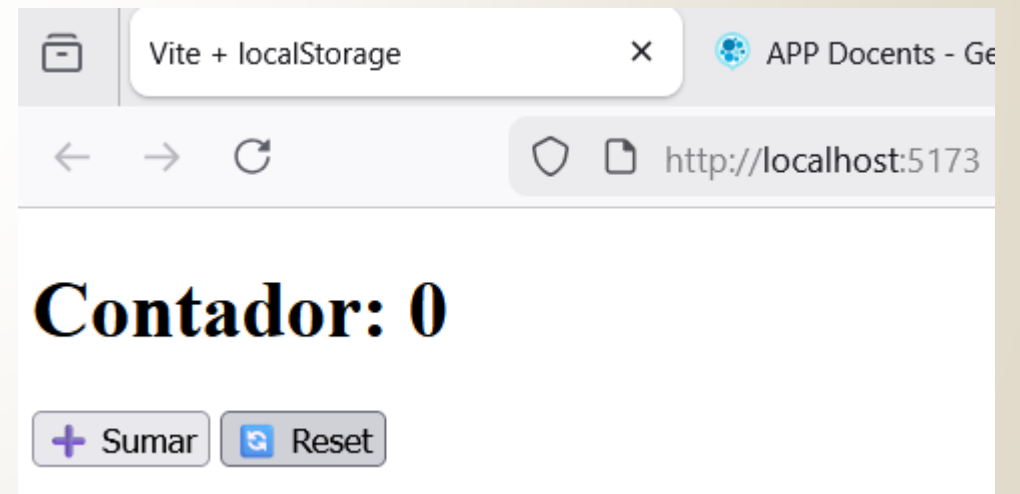
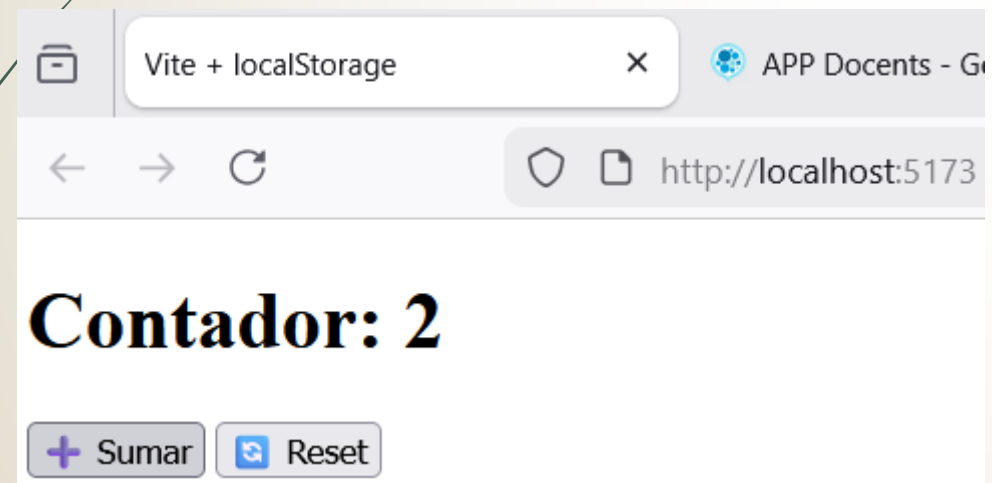
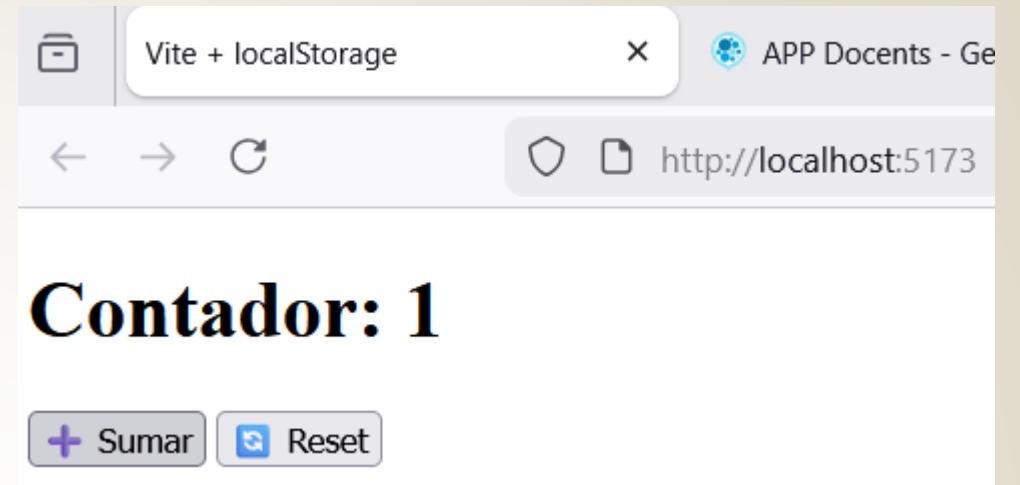
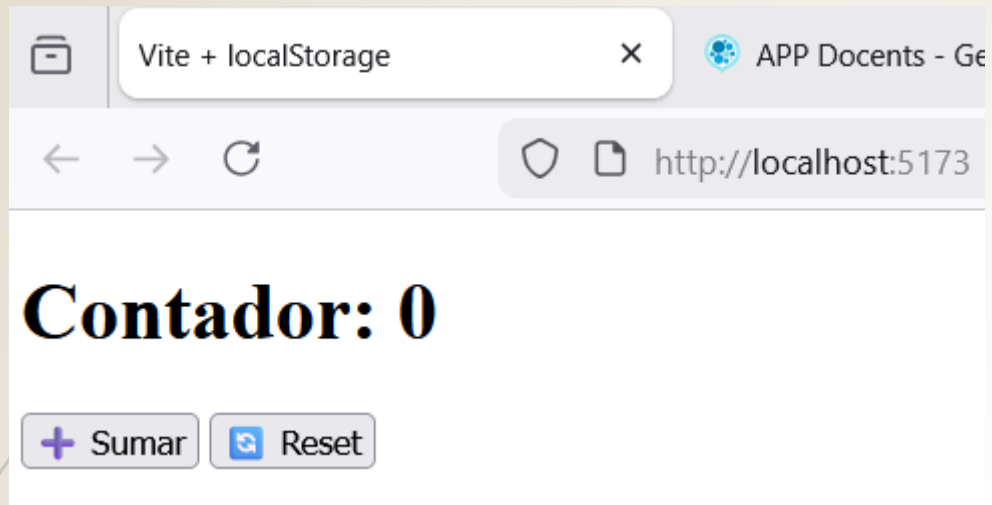
/src/main.js

... seguimos dentro de la función render() ...

```
document.querySelector("#sumar").onclick = () => {  
  contador++;  
  localStorage.setItem("contador", contador);  
  render();  
};  
  
document.querySelector("#reset").onclick = () => {  
  contador = 0;  
  localStorage.removeItem("contador");  
  render();  
};
```

... y la ejecutamos ...

render();



Cuándo **SÍ se usa localStorage** hoy en día ...

Muy común para **cosas simples, no críticas y para UX:**

- Preferencias de usuario
 - o tema claro/oscuro
 - o idioma
- Estado de UI
 - o sidebar abierta/cerrada
 - o filtros
- Formularios
 - o guardar borradores
- proyectos pequeños / MVPs (Minimum Viable Product)
- frontends con Vite, React, Vue, etc.

Cuándo **NO** se recomienda ...

Evítalo para:

- datos sensibles
 - o tokens JWT (JSON Web Token, transmisión de info de forma segura)
 - o contraseñas
- datos multi-usuario (acceso a la misma web desde dispositivos distintos o por distintos usuarios)
- datos que deben sincronizarse
- estado complejo de la app

... ya que es **síncrono** (puede bloquear el hilo), no tiene expiración, vulnerable a ataque **XSS** y tiene **límite** ~5MB

7.5.4. IndexedDB

Es una base de datos de tipo **NoSQL** implementada en todos los navegadores.

Almacena los datos como objetos, de forma similar a otros gestores de bases de datos como **MongoDB**.

Es una API muy potente que permite almacenar **grandes cantidades de datos estructurados** (objetos, arrays, etc.) directamente en el navegador, lo que la hace ideal para aplicaciones que requieren trabajar sin conexión (**offline**) o manejar datos complejos.

IndexedDB trabaja con operaciones asíncronas, usando eventos para procesar las respuestas.

IndexedDB “a pelo” es horrible de programar y ya casi nadie lo utiliza. Hoy en día se usa con librerías modernas como `idb`, `Dexie.js` o `localForage`.

Un ejemplo con **idb**, que es la más utilizada ...



```
import { openDB } from "idb";

const db = await openDB("app-db", 1, {
  upgrade(db) {
    db.createObjectStore("users", { keyPath: "id" });
  }
});

await db.put("users", { id: 1, name: "Ana" });
const user = await db.get("users", 1);
```

Cuándo **SÍ se usa IndexedDB** hoy en día ...

Muy común en **apps web modernas y grandes**:

- Mucha cantidad de datos
 - miles de registros
 - datos estructurados
- Apps offline / offline-first
 - PWAs
 - apps que funcionan sin conexión
 - sincronización cuando vuelve internet
- Cache avanzado del frontend
 - respuestas de APIs
 - resultados de búsquedas
 - catálogos grandes

- Apps tipo:
 - Gmail web
 - Google Docs
 - Spotify Web
 - Notion (Word + Google Docs + Trello)
 - Figma (Photoshop / Illustrator, pero en el navegador y varias personas editando a la vez)

→ Todas usan IndexedDB por debajo.

Cuándo **NO vale la pena** IndexedDB ...

No la uses para preferencias simples (tema, idioma), 2–3 valores pequeños, lógica trivial, ... (→ localStorage)

→ Regla práctica (la que usan equipos reales)

- UX / preferencias → localStorage
- datos grandes / cache / offline → IndexedDB
- datos oficiales y confiables → backend (bases de datos)

Más información:



https://developer.mozilla.org/es/docs/Web/API/IndexedDB_API

https://developer.mozilla.org/es/docs/Web/API/IndexedDB_API/Using_IndexedDB

7.5.5. Bases de datos

Base de datos	Tipo	Cuándo usar	Ejemplo
PostgreSQL	Relacional (SQL)	Apps serias, datos confiables, relaciones complejas	Gestión de usuarios y pedidos
MySQL	Relacional (SQL)	Proyectos simples o webs clásicas	Blog o página web con usuarios
MongoDB	NoSQL (documentos JSON)	Datos flexibles, MVPs, cambios rápidos	Notas, chats o publicaciones

7.5.6. Autenticación de usuarios

Las aplicaciones web que separan la parte cliente del servidor (ejecutándose en dominios o servidores web diferentes) no pueden utilizar cookies para que el cliente almacene información enviada al servidor en cada solicitud.

Por lo tanto, no es posible implementar una autenticación basada en sesiones de forma automática.

Es por ello que las API de servicios web en el servidor suelen usar **autenticación por token**.

La información de autenticación se suele mandar al servidor en una cabecera (Header) HTTP.

Lo estándar es usar la cabecera **Authorization**.

Se suelen utilizar 2 tipos de autenticación:

- **Autenticación básica:** Se envían en cada petición las credenciales del usuario como una cadena con formato: 'usuario:contraseña' codificada en base64 y con el prefijo **Basic**:

Si el usuario es `ana` y la contraseña `1234` :

1. Concatenas `usuario:contraseña` → `ana:1234`

2. Codificas en Base64 → `YW5hOjEyMzQ=`

La cabecera queda así ...

```
GET /perfil HTTP/1.1  
Host: ejemplo.com  
Authorization: Basic YW5hOjEyMzQ=
```

Ejemplo con fetch ...



```
const usuario = "ana";  
const contraseña = "1234";  
const credenciales = btoa(`${usuario}:${contraseña}`); // codifica en Base64  
  
fetch("https://ejemplo.com/perfil", {  
  headers: {  
    "Authorization": `Basic ${credenciales}`  
  }  
})  
  .then(res => res.json())  
  .then(data => console.log(data))  
  .catch(err => console.error(err));
```

- **Autenticación por token:** Cuando se envían (mediante un formulario de login) las credenciales, el servidor genera un token; por ejemplo, en formato **JWT** (JSON Web Token), y se lo envía como respuesta al cliente.

El token pasará a ser las credenciales del cliente y deberá enviarlo al servidor en cada petición, precedido por el prefijo **Bearer**:

```
// Supongamos que ya tenemos el token
const token = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...";

fetch("https://ejemplo.com/perfil", {
  headers: {
    "Authorization": `Bearer ${token}`
  }
})
.then(res => res.json())
.then(data => console.log(data))
.catch(err => console.error(err));
```


El proceso sería este ...

1. Usuario hace login con usuario/contraseña.
2. Backend responde con un token.
3. Cliente guarda el token.
4. Cliente envía token en cabecera `Authorization: Bearer <token>` en cada request.
5. Backend valida token → acceso concedido o denegado.

¿Qué es un **token**?

Es como una **llave digital** que permite que un usuario acceda a un recurso en un sistema.

- No contiene la contraseña del usuario.
- Sirve para identificar al usuario de forma segura.

El token suele contener información del usuario (**payload**), como su id, su rol y la fecha de expiración, todo ello firmado digitalmente con un algoritmo como por ejemplo HS256 para que no se pueda alterar la información y codificado en Base64 (**header-signature**).

A partir de su fecha de expiración habrá que renovarlo.

header

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

payload

```
{  
  "id": 123,  
  "usuario": "ana",  
  "rol": "admin",  
  "exp": 1710000000  
}
```

signature

```
header +  
payload +  
una clave secreta del backend
```

Para manejar el token desde JavaScript, lo almacenaremos en un sistema de almacenamiento local, como **LocalStorage**, una vez que recibamos la respuesta del inicio de sesión.

A partir de ese momento, lo más práctico será comprobar si existe un token almacenado cada vez que enviemos una petición HTTP al servidor y, en caso afirmativo, incluirlo en la solicitud:

```
const token = localStorage.getItem('token');  
if(token) headers.Authorization = 'Bearer ' + token;
```

Flujo de autenticación con JWT

[Usuario]

|
| Login: usuario + contraseña

v

[Frontend]

| POST /login

v

[Backend]

| Verifica credenciales
| Genera JWT:
| Header: {"alg":"HS256","typ":"JWT"}
| Payload: {id, usuario, rol, exp}
| Signature: firma secreta

v

[Frontend]

| Recibe JWT y lo guarda
| (localStorage / cookie httpOnly)

v

|

|

v

[Usuario hace request protegido]

| GET /perfil
| Header: Authorization: Bearer <JWT>

v

[Backend]

| Verifica token (signature + exp)
| Si es válido → concede acceso
| Si no → 401 No autorizado

v

[Frontend]

| Muestra datos protegidos al usuario

Ejemplo ...

→ login y esperamos a que el servidor nos devuelva el token



```
// src/login.js
async function login(usuario, password) {
  const res = await fetch("http://localhost:3000/login", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ usuario, password })
  });

  const data = await res.json();
  localStorage.setItem("token", data.token);
}
```

→ usaremos el token en peticiones protegidas

```
const token = localStorage.getItem("token");  
  
fetch("http://localhost:3000/perfil", {  
  headers: {  
    Authorization: `Bearer ${token}`  
  }  
});
```


→ en la parte servidor el token se generaría con la librería **firebase/php-jwt** → que instalamos con **composer** → que necesita tener **php** instalado



gestor de dependencias de PHP (como npm en JavaScript)

```
php -v
```

```
PHP 8.3.28
```

Instalamos composer desde aquí ...

<https://getcomposer.org/Composer-Setup.exe>

```
composer -v
```

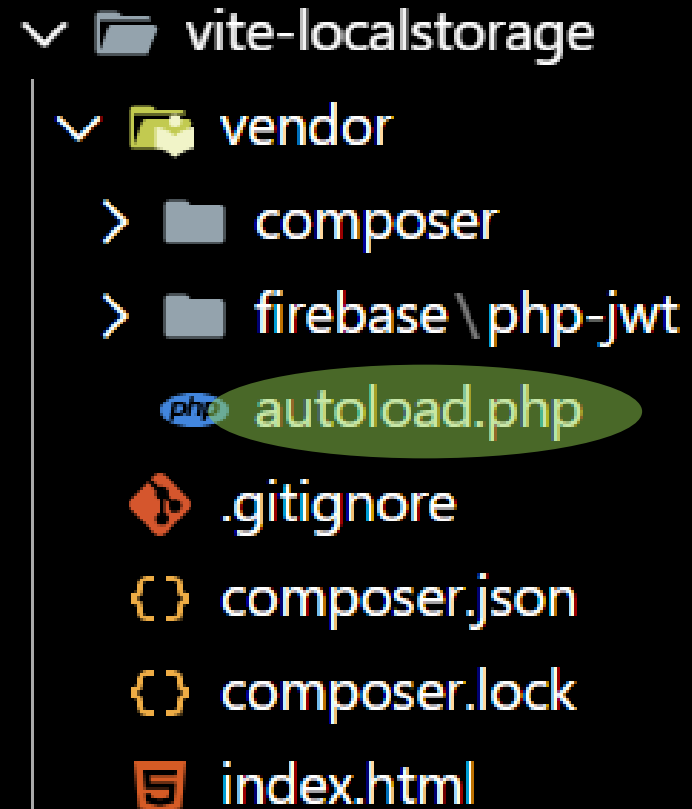
```
Composer
```

```
Composer version 2.9.4 2026-01-22 14:08:50
```

Instalamos la librería **firebase/php-jwt**

```
composer require firebase/php-jwt
```

```
Using version ^7.0 for firebase/php-jwt
```



```
▼ vite-localstorage
  ▼ vendor
    > composer
    > firebase\php-jwt
    php autoload.php
    .gitignore
    {} composer.json
    {} composer.lock
    index.html
```

El archivo **autoload.php** carga automáticamente todas las librerías que vayamos instalando para poder usarlas.

```
require 'vendor/autoload.php';
```

```
use Firebase\JWT\JWT;
```



Entorno de Desarrollo en VSCode

- >_ Instalar PHP

```
▶ php -v
```

- >_ Instalar Composer

```
▶ composer -V
```



Usuario → Frontend → Backend → JWT

Más información ...

<https://medium.com/@diego.coder/autenticaci%C3%B3n-en-node-js-con-json-web-tokens-y-express-ed9d90c5b579>

<https://medium.com/@anandam00/build-a-secure-authentication-system-with-nodejs-and-mongodb-58accdeb5144>

<https://medium.com/@it.ermias.asmare/user-authentication-and-authorization-in-express-and-mongodb-using-jwt-643503a23452>